ADMINISTRATION DES SYSTÈMES D'EXPLOITATION

SystExploit

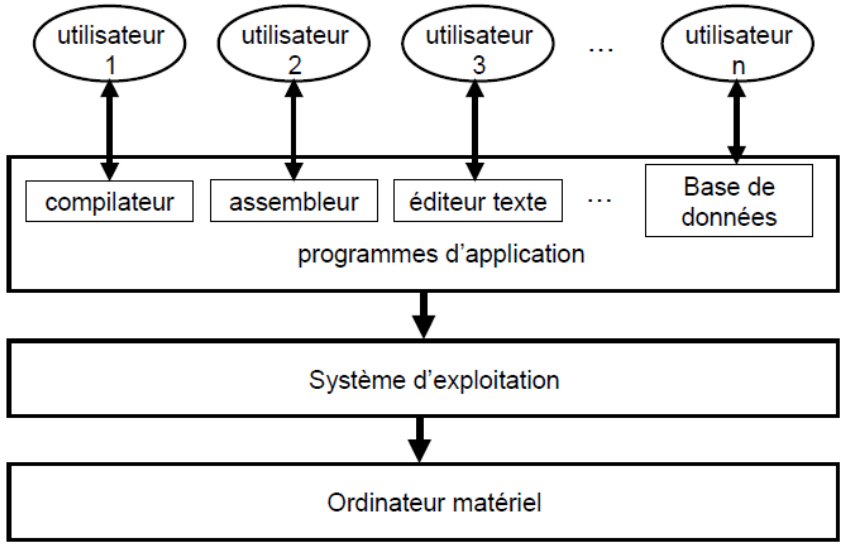
LES SYSTÈMES D'EXPLOITATION

Utilisateur = l'humain devant la machine
= suivant le contexte : utilisateur final, ou développeur

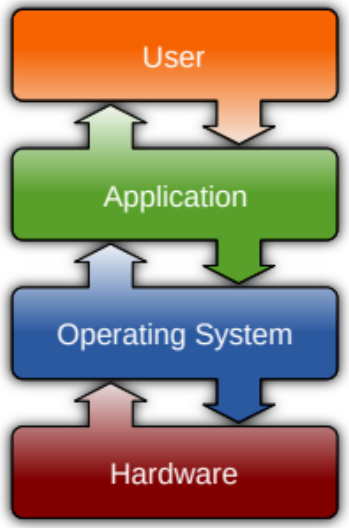
- C'est un **programme** :
  - pour les **utilisateurs** : doit leur permettre d'utiliser les fonctionnalités d'un ordinateur
  - doit aussi aider le **programmeur** à développer des logiciels de la façon la plus efficace possible
  - qui constitue donc une **interface** entre l'utilisateur et la machine physique

Applications = les logiciels avec lesquels veut interagir l'utilisateur final : messagerie, traitement de texte, lecteur de musique, etc.

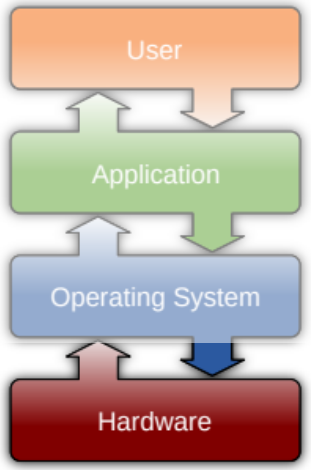
Matériel = la machine physique

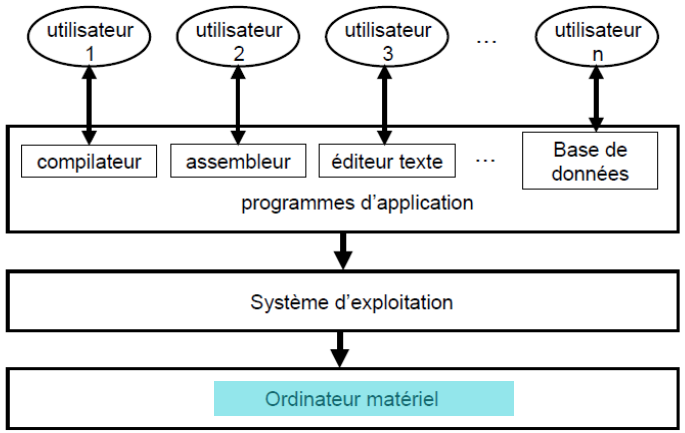


Les couches d'un système informatique

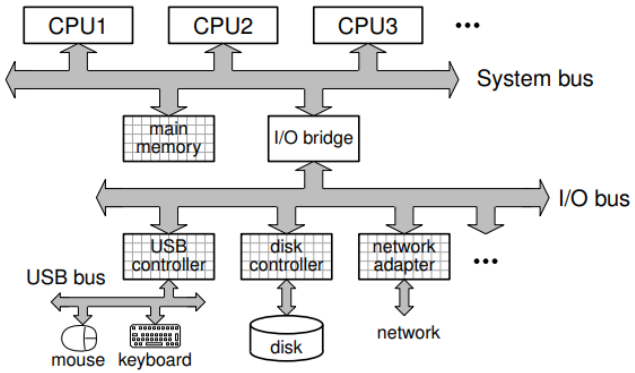


ORDINATEUR MATÉRIEL



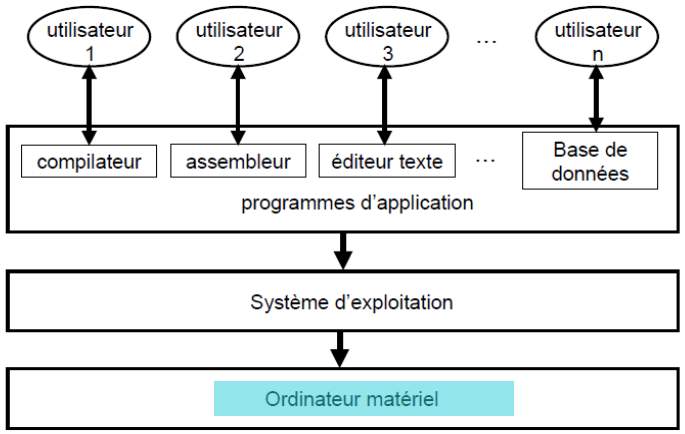


- **Processeur(s)** (un ou plusieurs) = CPU = UCT
  - exécution d'instructions machine d'un code en 3 étapes
    - chargement
    - décodage
    - exécution
  - La puissance du processeur se mesure en IPS (nb d'instruction par sec

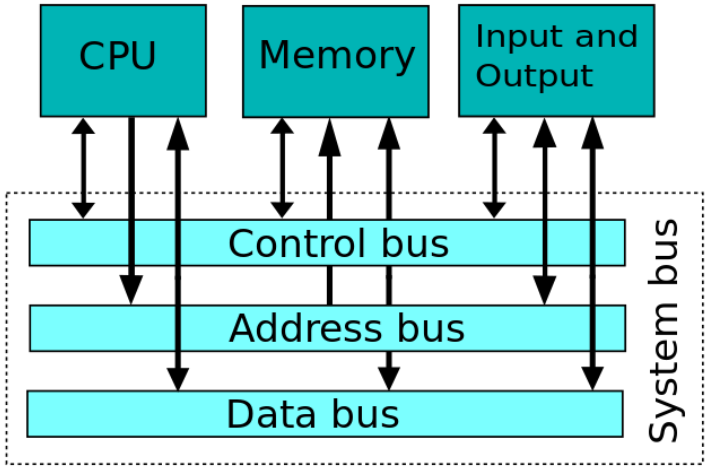


- **Mémoire centrale physique** (RAM)
  - qui sauvegarde temporairement :
    - les données à traiter
    - les instructions à exécuter
  - Elle s'organise comme un tableau d'octets
    - chaque octet a une adresse physique
    - un frame = page = plage contiguë d'adresses physiques qui dépend du SE (en général 4 096 octets)
  - Elles ont une capacité et un temps d'accès propres

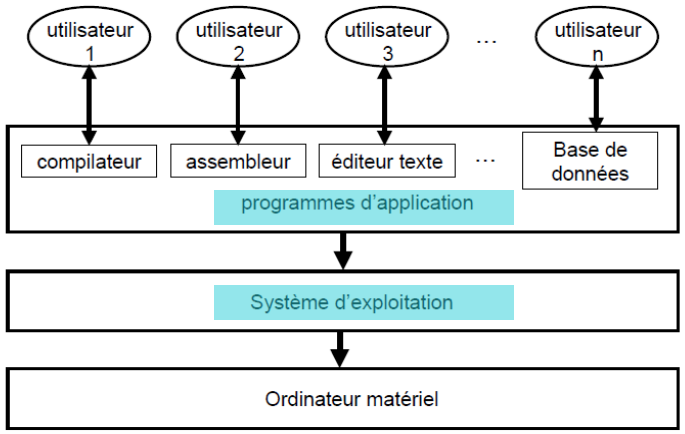
ORDINATEUR MATÉRIEL



- Les périphériques d'E/S (clavier, souris, écran, disques, cartes réseau, …)
  - permettent au processeur de communiquer = stocker et récupérer des informations à travers un bus
  - Bus :
    - Connecte les périphériques d'E/S et la mémoire au processeur
    - est constitué de :
      - lignes de données
      - lignes d'adresses
      - lignes de contrôle
    - Sa puissance se mesure par son **débit** (généralement en Mo/s)



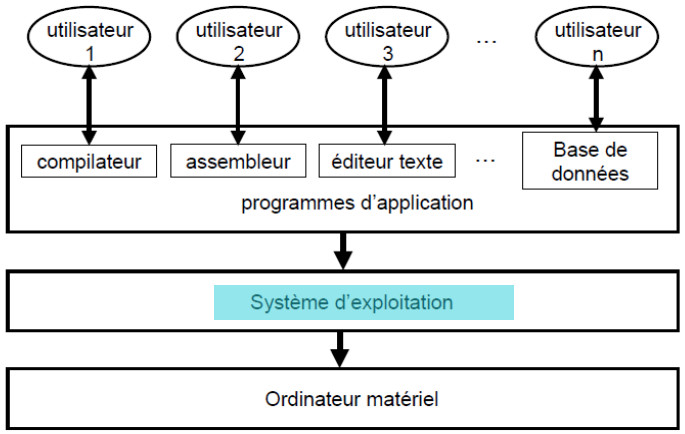
LOGICIELS



- **Programmes système**
  - Système d'exploitation
  - Utilitaires : compilateurs, éditeurs, interpréteurs de commandes, etc.

- **Programmes d'application** : résolvent des problèmes spécifiques des utilisateurs

LE SYSTÈME D'EXPLOITATION



- **Gère** et **contrôle** les composants de l'ordinateur
- Fourni une base appelée machine virtuelle à chaque utilisateur connecté au système qui cache la complexité sous une interface « plus jolie »
- Met à disposition des services pour les utilisateurs via des appels système
- Il permet ainsi à des utilisateurs de développer des applications sans se soucier des détails de fonctionnement et de gestion du matériel
  - exemple : sous Unix on peut lire un fichier avec un simple appel système read (fd, buf, 255) peut importe s'il se trouve sur HDD, SSD, Clé USB, clavier …

FONCTIONS DU SYSTÈME D'EXPLOITATION

- Chargement et lancement des programmes
- Gestion du matériel (processeurs, mémoire physique, bus et périphériques)
- Gestion des processus (programmes en cours d'exécution)
- Gestion des fichiers
- Protection contre les erreurs et détection des erreurs

INTERACTIONS UTILISATEUR/SYSTÈME

- Un utilisateur programmeur pourrait difficilement écrire des procédures lui-même sur le SE.

- Des bibliothèques de procédures appelées appels systèmes sont mis à sa disposition
  - exemple : sur un système Linux, un programme C peut utiliser des appels système comme open(), write(), read() pour réaliser des entrées-sorties de bas niveau.

- L'interpréteur de commandes est un exemple d'interface utilisateur/système.

L'INTERPRÉTEUR DE COMMANDES : SHELL

- est lancé dès la connexion au système sur les SE de la famille Unix/Linux
- Invite l'utilisateur à entrer une commande
- Le shell exécute cette commande grâce à des appels systèmes
- Il affiche le résultat ou les erreurs puis attend la commande suivante
- Exemple : la commande cat qui affiche le contenu d'un fichier

```
int main() {
    char buffer[100];
    int n;

    int fd=open("filename.txt", O_RDONLY);
    if(fd == -1)
        exit(1);

    n=read(fd, buffer, 100);
    while( n > 0) {
        write(STDOUT, buffer, n);
        n=read(fd, buffer, 100);
    }
    exit(0);
}
```

LE SHELL

- Les services offerts par un shell :
  - exécuter des programmes
  - charger un programme en mémoire, le lancer, l'arrêter
  - choisir quel programme est au premier-plan
  - explorer et administrer les espaces de stockage
  - naviguer dans les fichiers, copier, supprimer
  - confort et ergonomie
  - presse-papiers, drag-and-drop, corbeille
  - …

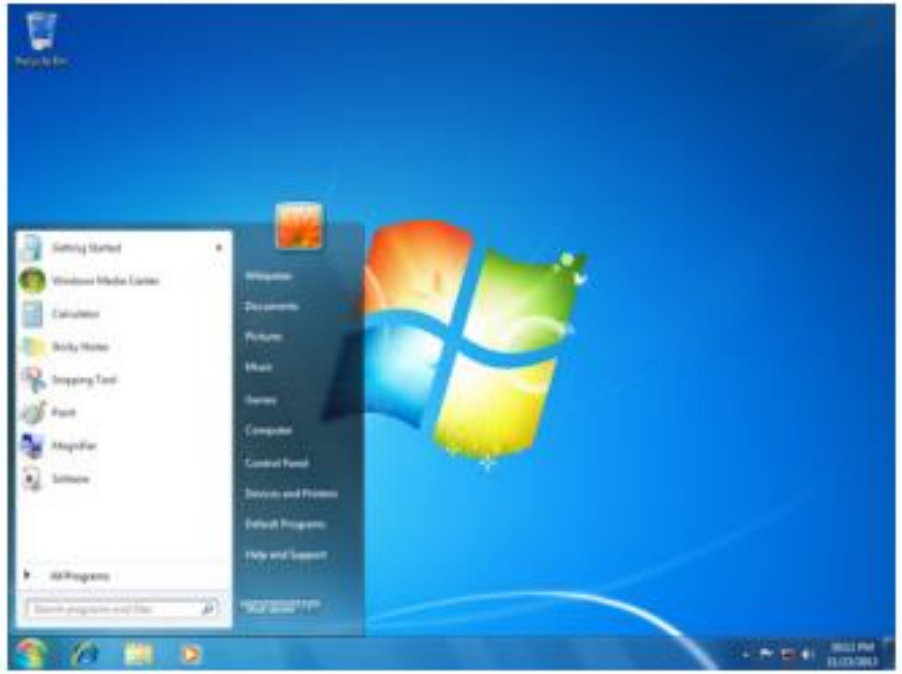
LE SHELL : INTERPRÉTEUR DE COMMANDE

- Interface textuelle = Command-Line Interface = CLI
- exemples : Bourne shell (1977), bash, zsh…

```
user1@iutm-inf-roja-p:~$ ls -l
total 0
-r-x-w--w- 1 user1 user1    0 Nov  1 20:03 essai
-rw-r--r-- 1 user1 user1    0 Nov  7 16:30 file25.txt
-rw-r--r-- 1 user1 user1    0 Nov  7 16:31 file.txt
drwxr-xr-x 1 user1 user1 4096 Nov  5 15:57 LPASR
drwxr-xr-x 1 user1 user1 4096 Oct 25 22:15 R305
user1@iutm-inf-roja-p:~$ cd LPASR/
user1@iutm-inf-roja-p:~/LPASR$ ls
cdm.csv  script_ip.sh
user1@iutm-inf-roja-p:~/LPASR$ cat cdm.csv
Année;Pilote;Nationalité;Écurie
2021;Verstappen;Pays-Bas;Red Bull-Honda
2020;Hamilton;Grande-Bretagne;Mercedes
2019;Hamilton;Grande-Bretagne;Mercedes
2018;Hamilton;Grande-Bretagne;Mercedes
2017;Hamilton;Grande-Bretagne;Mercedes
2016;Rosberg;Allemagne;Mercedes
2015;Hamilton;Grande-Bretagne;Mercedes
2014;Hamilton;Grande-Bretagne;Mercedes
2013;Vettel;Allemagne;Red Bull
```

LE SHELL : BUREAU GRAPHIQUE

- Interface graphique = Graphical User Interface = GUI
- exemples : Gnome, bureau de Windows, ...



LE SHELL : ECRAN DE SMARTPHONE

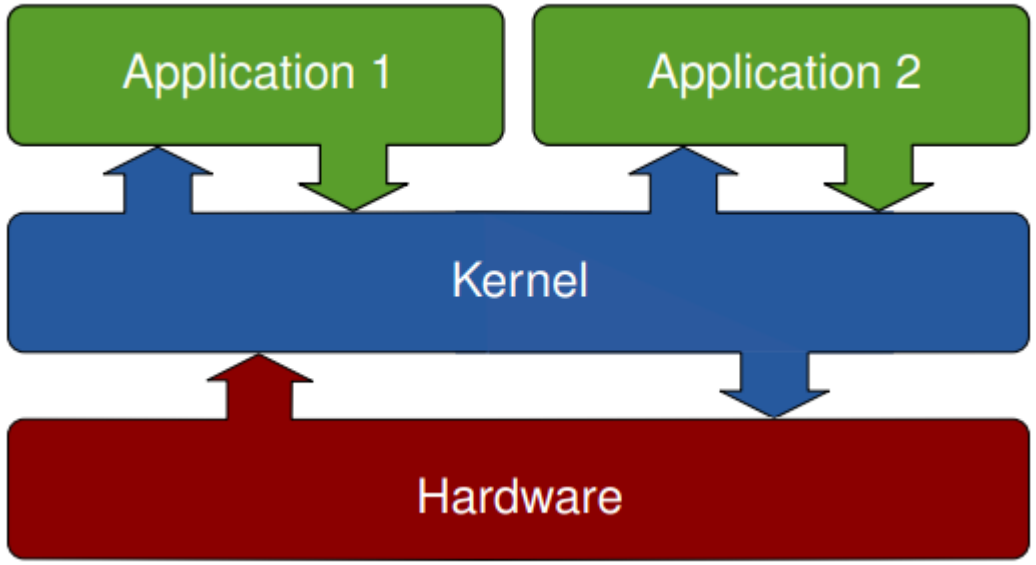
- exemples : Android Launcher, Google Now, ...

LE SHELL : EN RÉSUMÉ

- différents types de shell : CLI, GUI à souris, GUI tactile :
  - fonctionnalités similaires
  - pour l'OS : une application comme les autres
- votre OS contient volontiers des applications pré-installées …
  - shell, navigateur web, explorateur de fichiers, messagerie, lecteur multimedia, app store, etc …
- et aussi plein de «programmes système» :
  - développement : compilateur, assembleur, linker, etc
  - sécurité : antivirus, pare-feu, sauvegarde
  - maintenance : mise à jour, panneau de configuration
  - services réseau : web, base de données, accès distant
- Remarque : tous ces programmes sont des applications

L'OS C'EST QUOI ?

- Noyau = Kernel
- Le noyau c'est la partie de l'OS qui n'est pas une application



APPELS SYSTÈME

- Les processeurs ont 2 modes de fonctionnement afin de protéger le SE contre les intrusions ou erreurs :
  - Mode superviseur (maître) réservé au SE. Toutes les instructions sont autorisées.
  - Mode utilisateur (esclave) réservé aux programmes des utilisateurs. Certaines instructions ne sont pas permises.

- Un appel système est une interruption logicielle qui permet de :
  - passer en mode maître,
  - récupérer les paramètres et vérifier la validité de l'appel,
  - lancer l'exécution de la fonction demandée,
  - récupérer la (les) valeur(s) de retour
  - retourner au programme appelant avec retour au mode utilisateur.

LANGAGE DE PROGRAMMATION , LANGAGE MACHINE

```
00401be5 <main>:
  401be5:       55                      push   %rbp
  401be6:       48 89 e5                mov    %rsp,%rbp
  401be9:       48 83 ec 70             sub    $0x70,%rsp
  401bed:       be 00 00 00 00          mov    $0x0,%esi
  401bf2:       bf 10 20 48 00          mov    $0x482010,%edi
  401bf7:       b8 00 00 00 00          mov    $0x0,%eax
  401bfc:       e8 7f c5 03 00          callq  43e180 <__libc_open>
  401c01:       89 45 f8                mov    %eax,-0x8(%rbp)
  401c04:       83 7d f8 ff             cmpl   $0xffffffff,-0x8(%rbp)
  401c08:       75 0a                   jne    401c14 <main+0x2f>
  401c0a:       bf 01 00 00 00          mov    $0x1,%edi
  401c0f:       e8 7c 6a 00 00          callq  408690 <exit>
  401c14:       48 8d 4d 90             lea    -0x70(%rbp),%rcx
  401c18:       8b 45 f8                mov    -0x8(%rbp),%eax
  401c1b:       ba 64 00 00 00          mov    $0x64,%edx
  401c20:       48 89 ce                mov    %rcx,%rsi
  401c23:       89 c7                   mov    %eax,%edi
  401c25:       e8 86 c6 03 00          callq  43e2b0 <__libc_read>
  401c2a:       89 45 fc                mov    %eax,-0x4(%rbp)
  401c2d:       eb 30                   jmp    401c5f <main+0x7a>
  401c2f:       8b 45 fc                mov    -0x4(%rbp),%eax
```

APPELS SYSTÈME

- Les appels système manipulent des objets gérés par le SE parmi lesquels on trouve les processus et les fichiers.

- Exemples d'appels système

Appel système	Description
fork	Créer un processus
wait, waitpid	Attendre la terminaison d'un processus fils
execve	remplacer le programme en cours par un autre programme
exit	Terminer l'exécution
open	Créer ou ouvrir un fichier
close	Fermer un fichier
read	Lire des données à partir d'un fichier
write	Ecrire des données à partir dans un fichier
lseek	Potionner le pointeur d'un fichier
stat	Obtenir l'état des attributs d'un fichier
mkdir	Créer un répertoire
rmdir	Éliminer un répertoire
link	Ajouter un chemin d'accès à un fichier
unlink	Supprimer un chemin d'accès à un fichier
mount	Monter un système de fichiers
umount	Démonter un système de fichiers
chdir	Changer de répertoire
chmod	Changer les permissions d'accès
kill	Envoyer un signal à un processus
time	Obtenir le temps d'exécution d'une commande

LES PROCESSUS

- C'est un programme en cours d'exécution
- Les appels système permettent :
  - la création d'un processus
  - l'arrêt normal ou anormal d'un processus
  - La synchronisation ou la communication entre processus

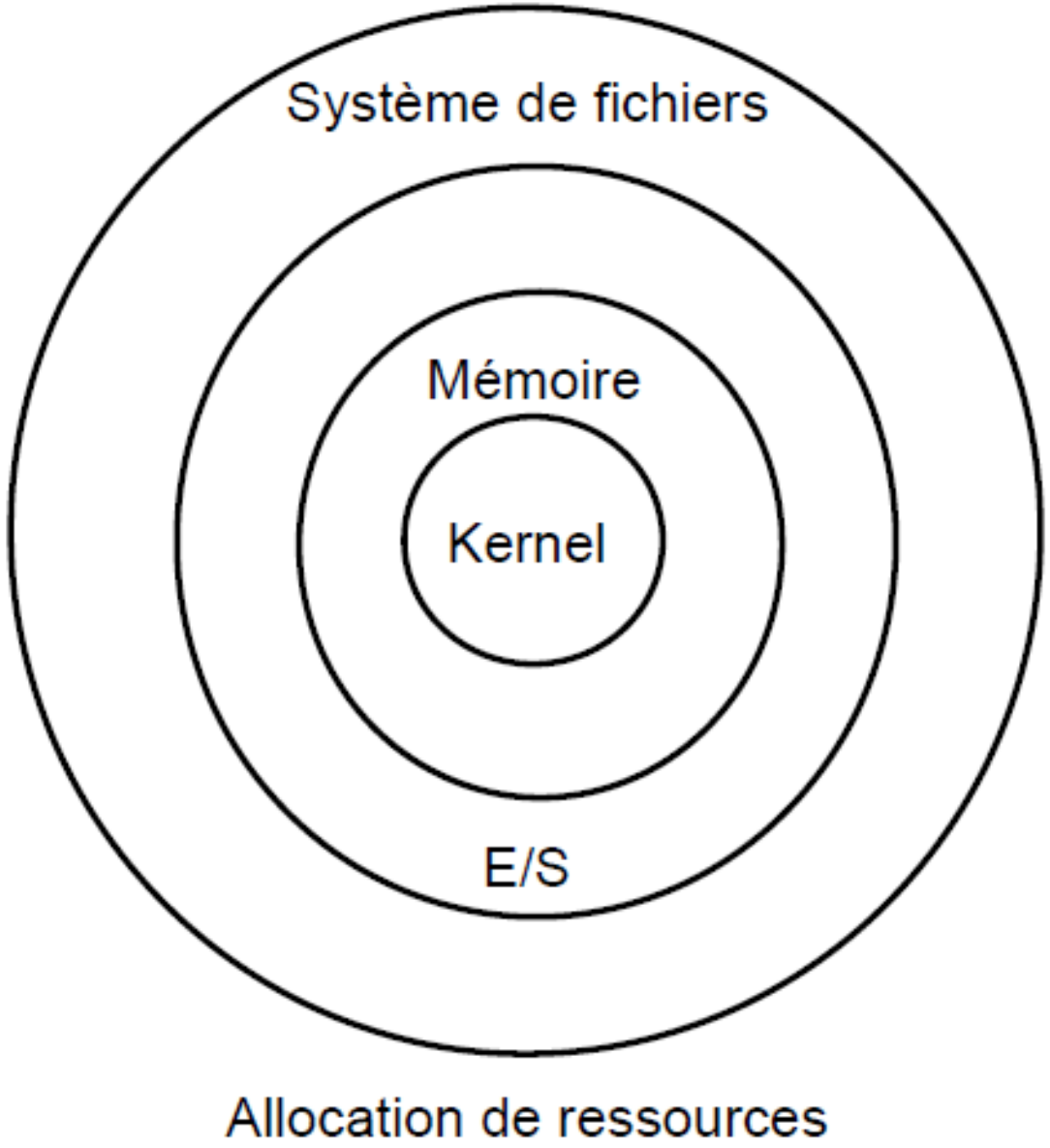
- Un processus peut créer un ou plusieurs processus fils qui peuvent aussi créer des processus fils.
- On a alors une structure arborescente de processus
- Les appels systèmes permettent aussi à des processus de créer des processus légers nommés threads

LES FICHIERS

- Un fichier ordinaire est un ensemble de données sauvegardées dans une unité de stockage (ex. : disque)
- Un fichier spécial représente un périphérique (clavier, écran, tube de communication, etc.)
- Les fichiers peuvent être regroupés dans des répertoires
- L'accès aux fichiers se fait en donnant un chemin d'accès :
  - Relatif
  - absolu

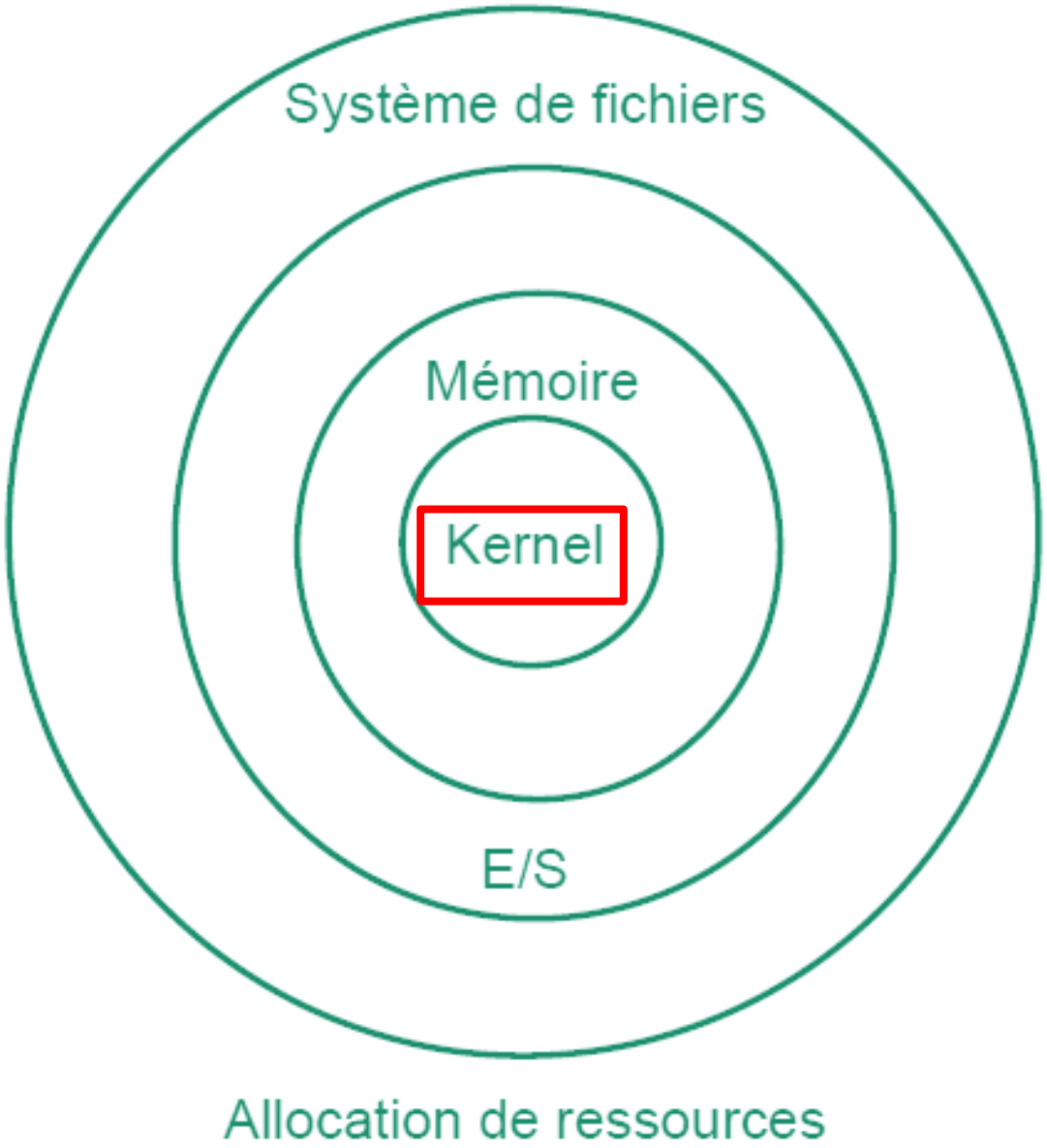
STRUCTURE D'UN SE

- Il est structuré en couches
- Chaque couche utilise les fonctions des couches inférieures



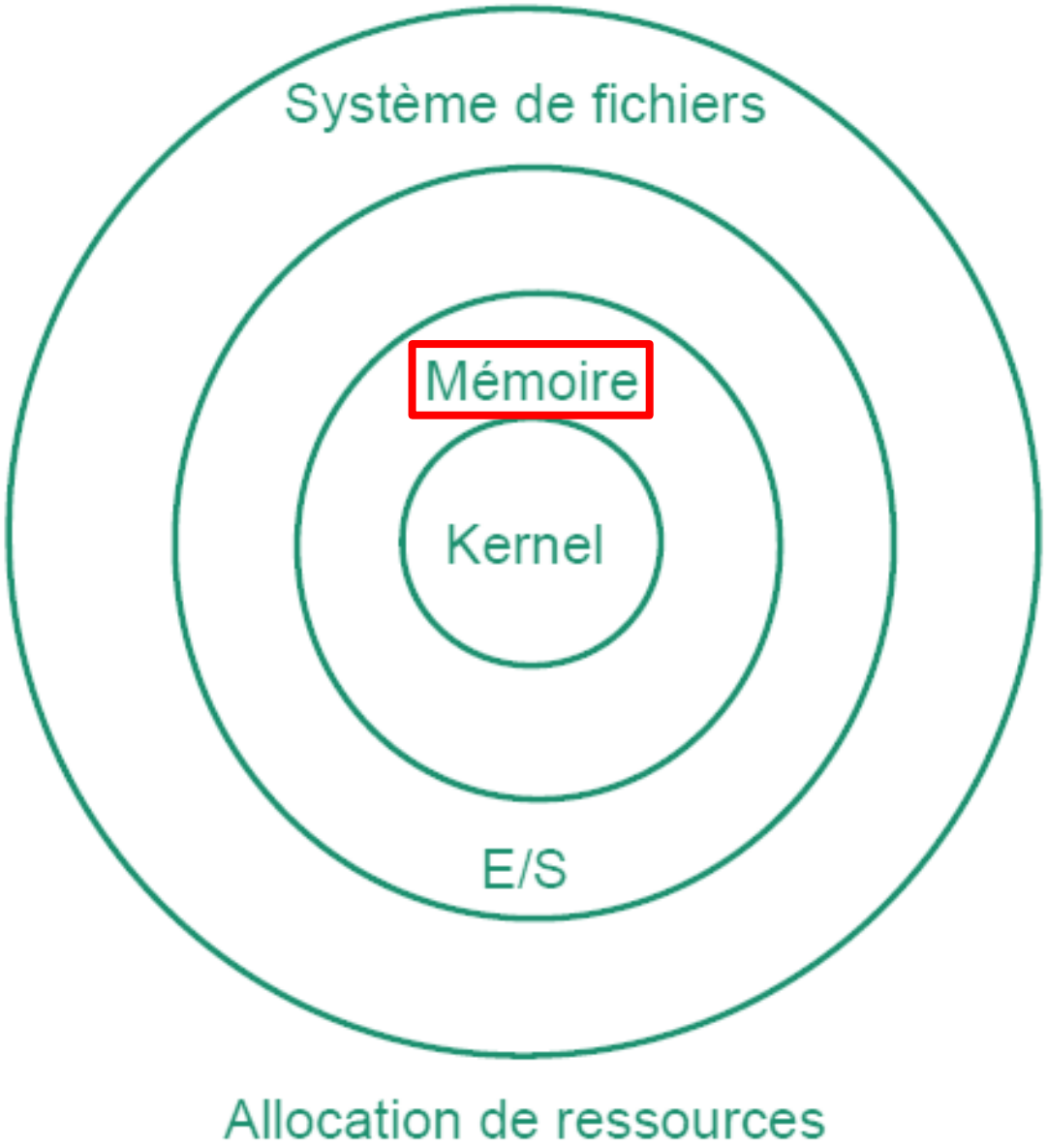
STRUCTURE D'UN SE

- Au plus bas niveau on trouve le **noyau**, l'interface entre le matériel et le logiciel. Il se charge, en utilisant les fonctions fournies par le matériel, de gérer l'UCT, les interruptions et les processus. Il doit entièrement résider en mémoire.



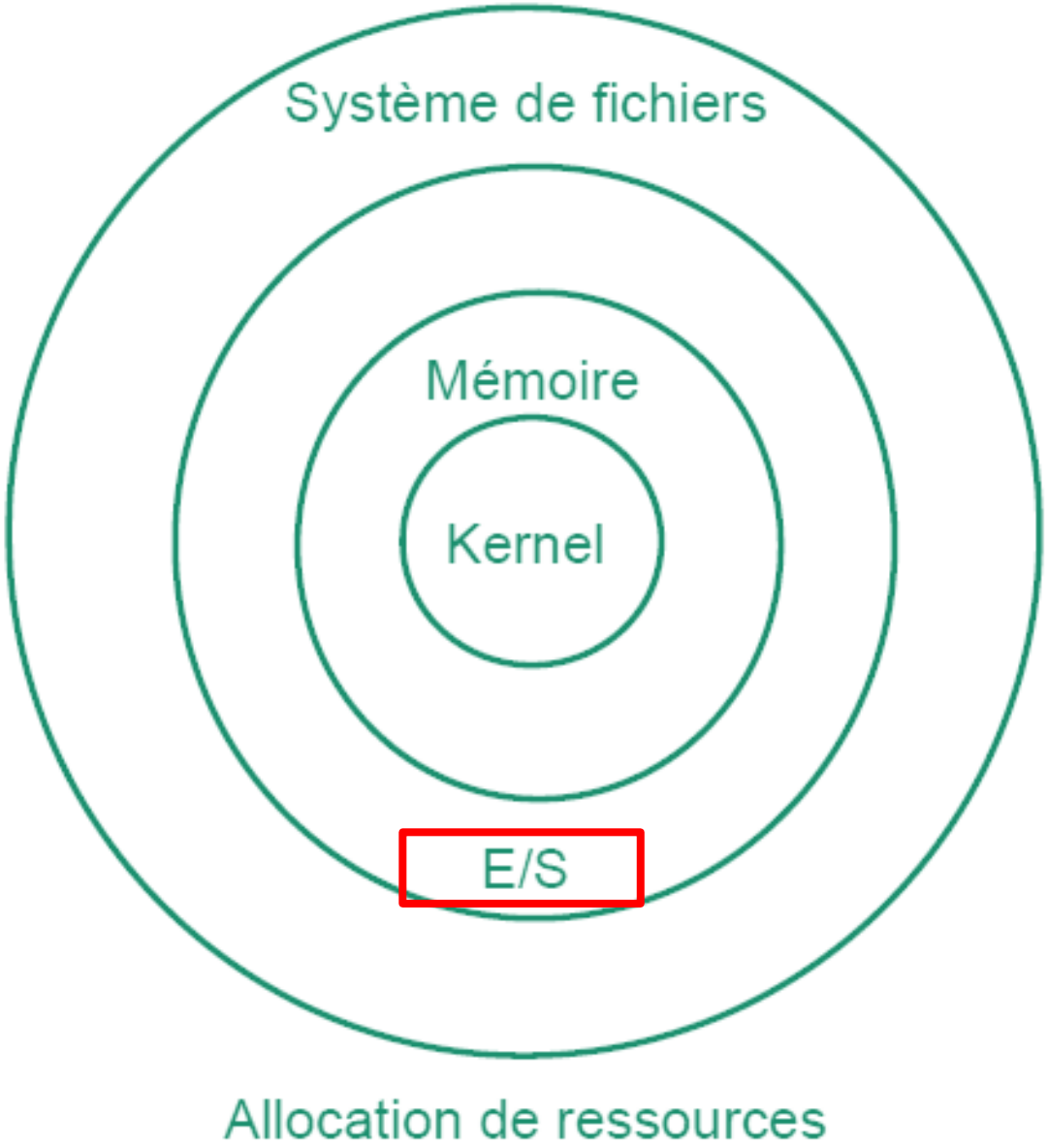
STRUCTURE D'UN SE

- Au second niveau, on trouve le gestionnaire de la mémoire qui se charge du partage de la mémoire entre les processus en attente d'exécution.



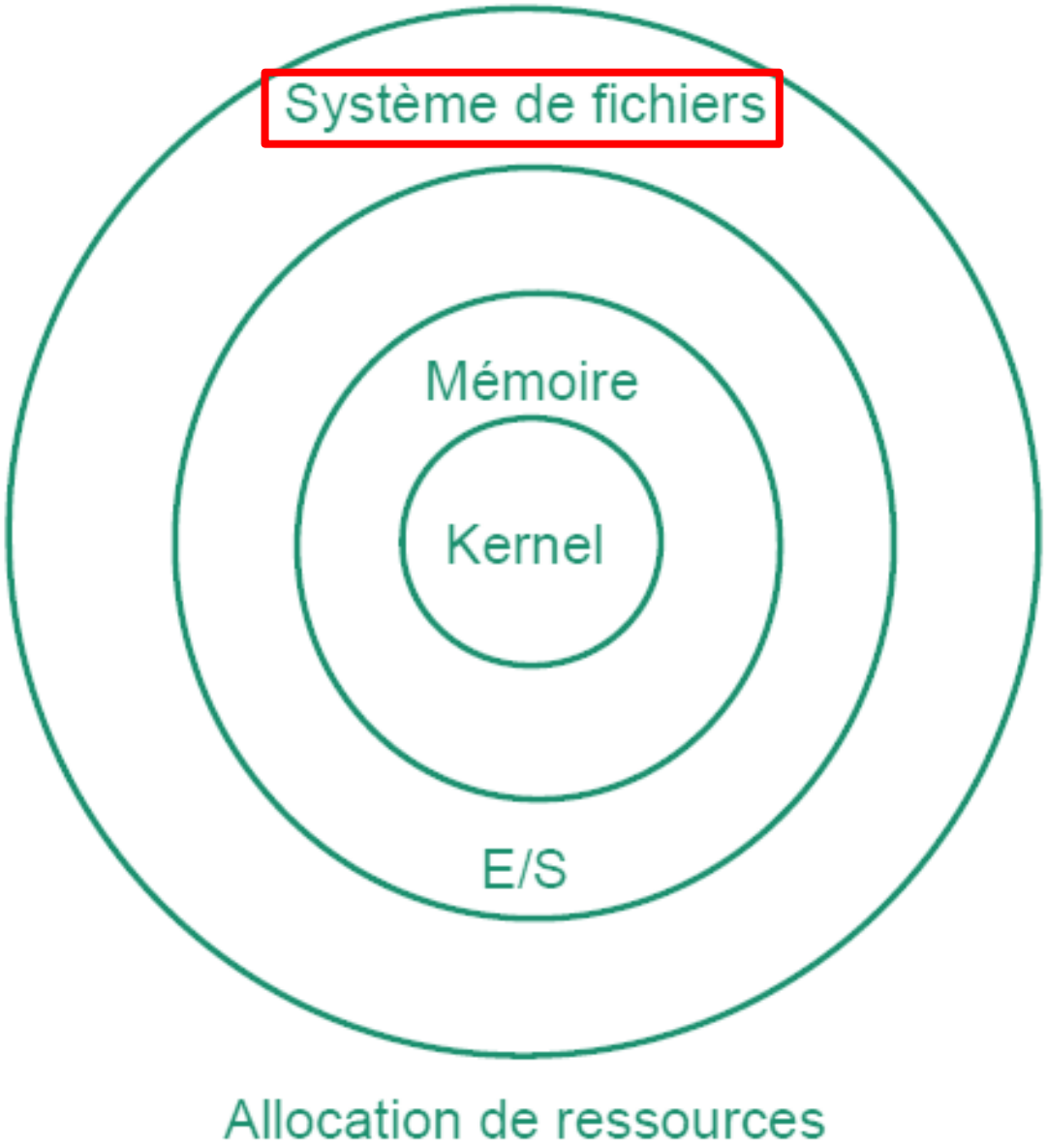
STRUCTURE D'UN SE

- Au troisième niveau, on a le module de gestion des entrées/sorties qui se charge de gérer tous les périphériques (clavier, écran, disques, imprimantes, etc.)



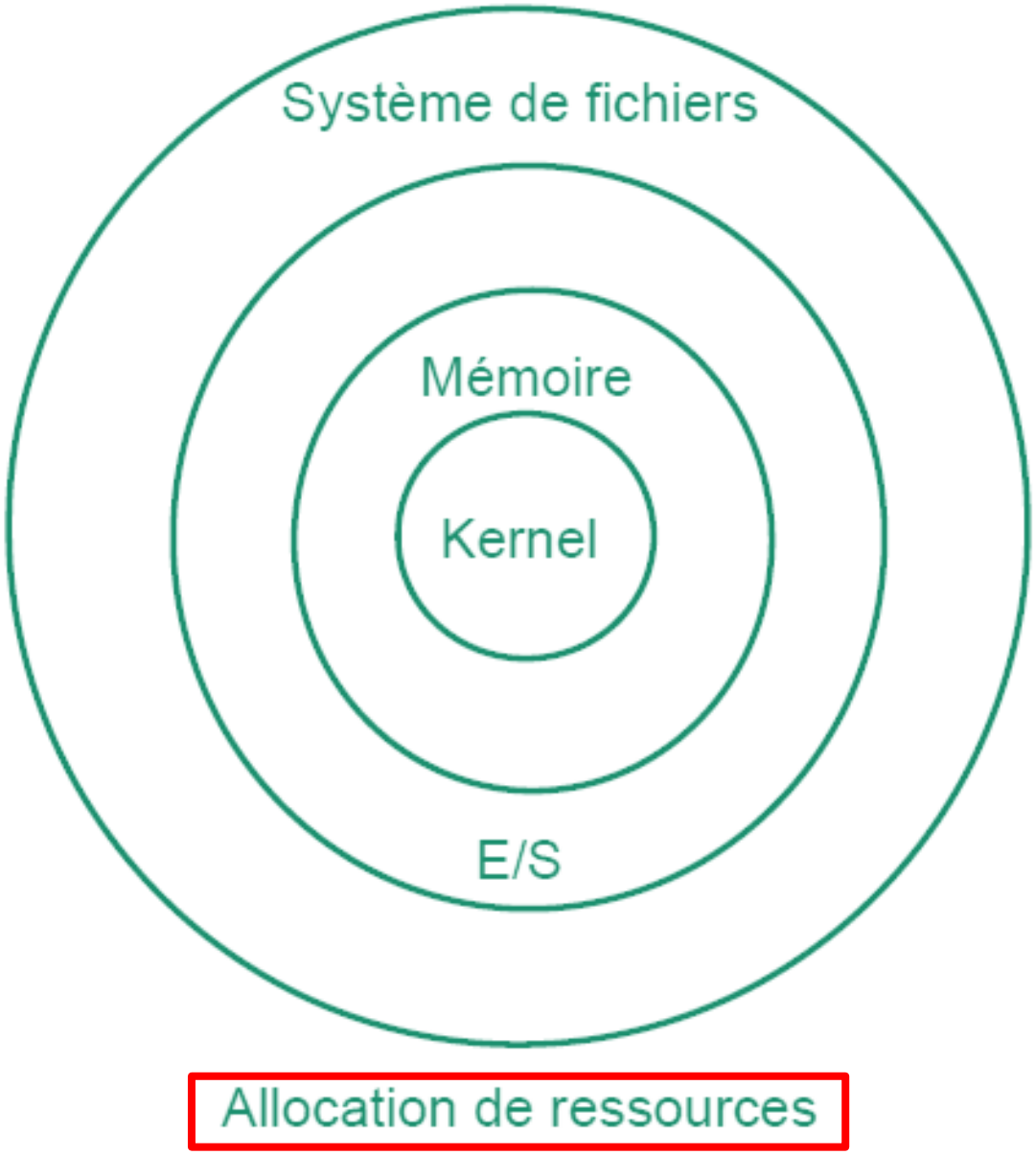
STRUCTURE D'UN SE

- Au quatrième niveau, on trouve le gestionnaire de fichiers qui se charge de la gestion de l'espace du disque, de la manipulation des fichiers tout en assurant l'intégrité des données, la protection des fichiers, etc.



STRUCTURE D'UN SE

- Au cinquième niveau, on a le module d'allocation de ressources qui se charge :
- d'assurer une bonne utilisation des ressources
- de comptabiliser et de fournir des statistiques sur l'exploitation des ressources principales
- de créer de nouveaux processus et leur attribuer un niveau de priorité
- de permettre à chaque processus existant dans le système d'obtenir les ressources nécessaires dans des limites de temps raisonnables
- d'exclure mutuellement les processus qui demandent une ressource non partageable et d'éviter les situations de blocage



LE SYSTÈME DE FICHIERS ET LES PÉRIPHÉRIQUES D'E/S

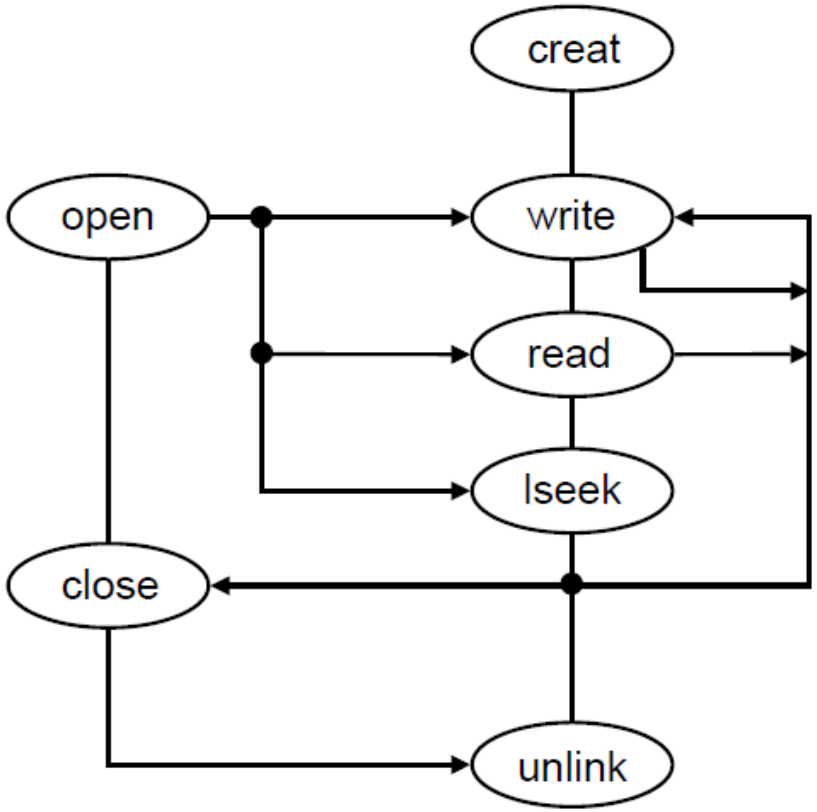
- C'est la partie du système d'exploitation qui se charge de gérer les fichiers :
  - leur création (identification, allocation d'espace sur disque),
  - leur suppression,
  - leur accès en lecture et en écriture,
  - le partage de fichiers et leur protection en contrôlant les accès.

- Pour le SE : 1 fichier = 1 suite d'octets, répartis dans des blocs de données de même taille
- Le fichier est identifié par un nom et possède des attributs qui servent à le localiser, à contrôler les accès, à le lire et décrire son contenu

CYCLE DE VIE D'UN FICHIER

- Les fichiers ont un cycle de vie. Ils sont :
  - créés ou ouverts,
  - lus
  - modifiés
  - supprimés

- Illustration du cycle de vie d'un fichier et appels système associés

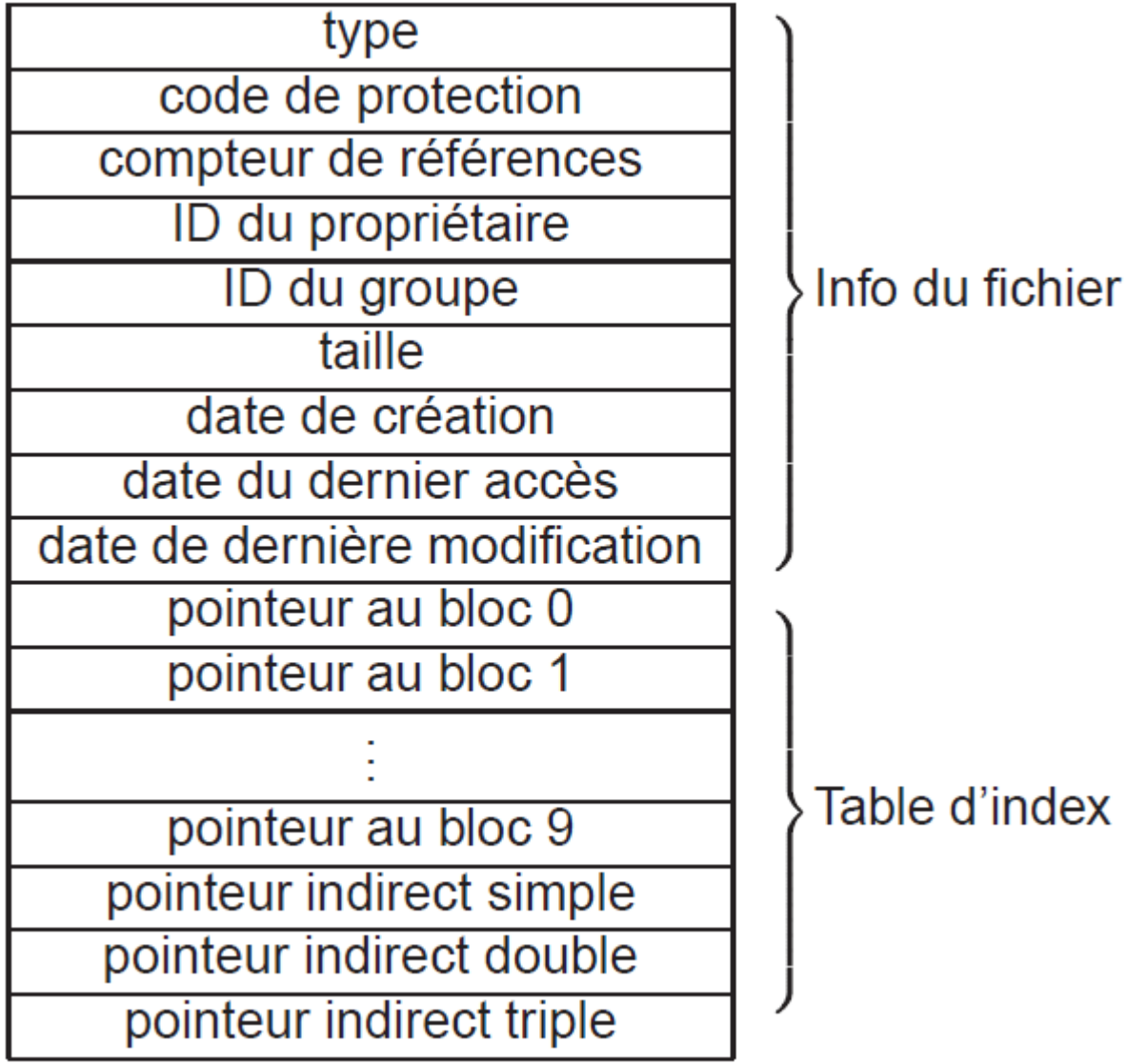


ACCÈS AUX FICHIERS

- Les fichiers d'une unité de stockage sont regroupés dans des répertoires qui peuvent contenir d'autres répertoires.
- Les attributs de fichiers peuvent être regroupés en 3 catégories :
  - attributs de contrôle d'accès : code de protection, mot de passe, propriétaire
  - attributs de type et d'état courant : indicateur du type ASCII ou binaire, taille courante, date de création, date de dernière modification, …
  - attributs permettant de localiser les données du fichier
- Ces attributs sont rassemblés dans une structure appelée nœud d'information, i-nœud, **i-node**.

I-NODES

- L'i-noeud contient les informations suivantes :
- le type du fichier (régulier, répertoire, caractère, bloc ou tube)
- le code de protection sur 9 bits
- l'identificateur et groupe du propriétaire
- les dates de création, du dernier accès et de la dernière modification
- le compteur de références
- la taille
- la table d'index composée de 13 numéros, de blocs et de pointeurs



APPELS SYSTÈME POSIX SUR LES FICHIERS

- Les systèmes fournissent un ensemble d'appels système. Dans le cas d'Unix/Linux, les principaux appels système Posix relatifs aux fichiers sont :
  - open() et creat() pour l'ouverture d'un fichier
  - close() pour la fermeture d'un fichier
  - read() pour la lecture d'un fichier
  - write() pour l'écriture dans un fichier
  - lseek() pour déplacer le pointeur de fichier
  - stat() pour récupérer des informations d'un fichier
  - link() pour créer un autre chemin d'accès à un fichier physique
  - unlink() pour supprimer chemin d'accès à un fichier physique

OUVERTURE D'UN FICHIER

```
#include <unistd.h>
#include <fcntl.h>
int open(char * filename, int mode);
int open(char *filename, int mode, int permission);
int creat(char * filename, int permission);
```

- En cas de succès, open() retourne un descripteur du fichier.
- En cas d'échec, il retourne 0.
- Chaque fichier ouvert a un pointeur qui pointe sur un élément du fichier.
- Ce pointeur sert à parcourir le fichier.
- A l'ouverture du fichier, le pointeur de fichier pointe sur le premier élément du fichier, sauf si l'option O_APPEND a été spécifiée.
- Les lectures et les écritures se font à partir de la position courante du pointeur.
- Chaque lecture et chaque écriture entraînent la modification de la position du pointeur.
- L'appel système creat() 1 est de moins en moins utilisée. Son usage est équivalent à celui de :

  open(char *filename, O_WRONLY|O_CREAT|O_TRUNC, int mode)

OUVERTURE D'UN FICHIER : MODE

```
#include <unistd.h>
#include <fcntl.h>
int open(char * filename, int mode);
int open(char *filename, int mode, int permission);
int creat(char * filename, int permission);
```

- Le mode est une combinaison de plusieurs éléments assemblés par un OU logique. Il faut utiliser obligatoirement l'une de trois constantes :
  - O_RDONLY: fichier ouvert en lecture exclusive.
  - O_WRONLY: fichier ouvert en écriture exclusive.
  - O_RDWR: fichier ouvert en lecture et en écriture.
- On peut ensuite utiliser d'autres constantes qui permettent de mieux préciser l'usage :
  - O_CREAT: créer le fichier ouvert même en lecture exclusive.
  - O_EXCL: employé conjointement avec O_CREAT, cette constante garantit qu'on n'écrasera pas un fichier déjà existant. L'ouverture échoue si le fichier existe déjà.
  - O_TRUNC: utilisée avec O_WRONLY ou avec O_RDWR, permet de ramener la taille d'un fichier existant à zéro.

OUVERTURE D'UN FICHIER : PERMISSIONS

- Les **permissions** sont utilisées lors de la création d'un fichier. Elles servent à signaler les autorisations d'accès au nouveau fichier crée. On peut les fournir en représentation octale directement (précédées d'un 0) ou bien on peut utiliser les constantes symboliques.
- Exemple : "S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH" = 0644 = rw-r--r--
  On donne les droits de lecture à tous et les droits de lecture et écriture seulement au propriétaire du fichier.

Constante symbolique	Valeur octale	Signification de l'autorisation
S_IRUSR	00400	Lecture pour le propriétaire
S_IWUSR	00040	Écriture pour le propriétaire
S_IXUSR	00100	Exécution pour le propriétaire
S_IRWXU	00700	Lecture + écriture + exécution pour le propriétaire
S_IRGRP	00020	Lecture pour le groupe
S_IROTH	00004	Lecture pour tout le monde
S_IWOTH	00002	Écriture pour tout le monde
S_IXOTH	00001	Exécution pour tout le monde

EXEMPLE D'OUVERTURE

- Cet appel système crée, puis ouvre un fichier nommé fichier pour un accès en lecture/écriture
- Il le crée s'il n'existe pas
- Le descripteur de fichier est fd
- Le code de protection du fichier est 0600 ce qui correspond à wr-------

```
int fd;
fd = open("fichier", O_CREAT | O_RDWR, 0600);
```

FERMETURE DES FICHIERS

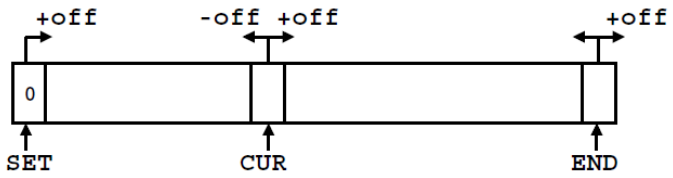
- L'appel système close() permet de libérer un descripteur de fichier fd.
- Le fichier pointé par le descripteur libéré est fermé s'il n'a plus aucun descripteur associé au fichier.

```
int close (int fd);
```

ACCÈS DIRECT À UNE POSITION DANS LE FICHIER

int lseek(int fd, int offset, int origine);

- L'appel système lseek() permet de modifier la position courante du pointeur de parcours du fichier
- Il retourne la nouvelle position du pointeur ou -1 en cas d'erreur
- La nouvelle position dépend des paramètres offset et origine
  - SEEK_SET: Position = offset
  - SEEK_CUR: Position courante + offset
  - SEEK_END: Taille du fichier + offset



- Ainsi, on peut récupérer la position courante du pointeur de parcours d'un fichier, par ce simple appel :

lseek(fd,0,SEEK_CUR) où fd est son descripteur de fichier

EXEMPLE 2

- Soit le fichier fichier contenant la chaîne : "le printemps".
- A l'ouverture de ce fichier, le pointeur de fichier est positionné sur le 1er caractère "l". Sa valeur est 0.

```
char buf ;

//déplacer le pointeur du fichier à la position 3 du fichier
lseek(fd, 3,SEEK_SET);

// la valeur du pointeur est 3.
// lire un caractère dans buf (buf reçoit p)
read(fd, buf, 1);

// positionner le pointeur deux caractères plus loin
lseek(fd,2,SEEK_CUR);

// la valeur du pointeur est 4+2.
// lire un caractère dans buf (buf reçoit n)
read(fd,buf,1);

//positionner le pointeur à la fin du fichier
lseek(fd,0,SEEK_END);
read(fd,buf,1) ; // retournera 0
```